

Simulador de Proof of Work

Fundamentos de Blockchain · Trabajo intermedio · 1 de octubre de 2026

Propósito: préstamos de equipo de laboratorio

La aplicación registra préstamos y devoluciones de equipos como microscopios, osciloscopios y multímetros. Cada movimiento guarda el equipo, la acción y el responsable. El remitente es la llave pública de quien firma; la transacción también contiene el propósito y la hora en UTC. Así se puede revisar quién autorizó cada movimiento y detectar cambios posteriores en el registro.

La regla propia del proyecto impide prestar un equipo que ya tiene un préstamo activo. Para devolverlo, deben coincidir la llave que firmó el préstamo y el nombre del responsable. Esta regla se comprueba al registrar la transacción, antes de minar y al validar la cadena. Una firma correcta demuestra autoría, pero por sí sola no garantiza que un préstamo sea permitido.

Decisiones de diseño

Cada billetera genera una llave privada `Ed25519`. La llave pública y la firma se representan en hexadecimal. La firma cubre la transacción completa serializada con `json.dumps(..., sort_keys=True)`. La llave privada permanece en el servidor. La aplicación vuelve a verificar la firma antes de lanzar los mineros y rechaza firmas alteradas.

Un bloque contiene número, nonce, transacción, firma, `hash_anterior`, minero y hash. SHA-256 se calcula con todos los campos excepto hash. La serialización ordenada permite recalcularlo de manera determinista. El campo minero queda incluido para que cambiar al destinatario de la recompensa invalide el hash. El génesis tiene número cero, hash anterior de 64 ceros y hash calculado; no exige firma ni minería porque no registra una transacción.

La carrera usa cuatro hilos en el mismo proceso. El nodo `i` recorre los nonces `i + 4k`: por ejemplo, el nodo 1 prueba 1, 5, 9 y 13. Cada nodo trabaja sobre una copia propia del bloque, con su nombre como minero. Aunque los rangos de nonces son disjuntos, los nodos no calculan el mismo mensaje porque su campo minero es diferente.

Un Event compartido señala el fin de la carrera. Un RLock protege la elección del ganador: dentro del candado se comprueba otra vez si alguien ya ganó, se agrega un único bloque, se acreditan 50 unidades y se retira la transacción pendiente. El estado minando pasa a falso cuando todos los hilos han terminado. Esto evita mostrar una carrera finalizada mientras aún se ejecuta algún trabajador.

Interfaz y alcance

Flask ofrece las rutas `/`, `/transaccion`, `/minar` y `/estado`. La página consulta el estado cada 300 ms para mostrar intentos, último hash, nonce y saldo por nodo. Al terminar, recarga la cadena y marca al ganador con un trofeo. Las billeteras, los saldos y la cadena viven en memoria; reiniciar el servidor crea un estado nuevo. Es un simulador local sin red P2P ni dinero real.

Mediciones y validación

Método de medición

Se ejecutaron 5 carreras independientes para cada dificultad, con cuatro nodos, Python 3.12.14 y Windows 11. Cada carrera generó una billetera y una transacción nuevas. Se usó `time.perf_counter` desde el inicio de la minería hasta terminar los cuatro hilos. La firma se hizo antes del cronómetro y la cadena se verificó después. Los valores originales y el entorno se guardan en `mediciones.csv` y `mediciones.json`.

Dificultad	Carreras	Promedio (s)	Mín. / máx. (s)	Intentos promedio
3	5	0.049	0.002 / 0.119	3,048.0
4	5	1.061	0.178 / 3.272	63,114.6
5	5	10.248	1.990 / 23.947	540,712.2

Intentos = suma de hashes probados por los cuatro nodos, incluido el hash ganador. El tiempo incluye coordinación y finalización de los hilos.

Por qué cada cero multiplica el trabajo esperado por 16

Un dígito hexadecimal puede tomar 16 valores. Si los resultados de SHA-256 se modelan como uniformes, la probabilidad de que un hash comience con d ceros es $p = 16^{-d}$. El número de intentos hasta el primer éxito tiene distribución geométrica y su media es $1/p = 16^d$. Por eso se esperan 4,096 intentos para $d = 3$; 65,536 para $d = 4$; y 1,048,576 para $d = 5$. Agregar un cero divide la probabilidad por 16 y multiplica por 16 los intentos esperados.

El factor describe una expectativa, no una duración fija. En esta muestra, el promedio de tiempo creció aproximadamente 21.69 veces de 3 a 4 y 9.65 veces de 4 a 5. Cinco carreras son pocas para estimar con precisión una variable de tanta dispersión. También intervienen la carga del equipo, el costo de serializar, la coordinación y el GIL de CPython: cuatro hilos no garantizan cuatro veces más velocidad. Los resultados medidos se conservan tal como ocurrieron.

Pruebas y demostración

Las pruebas automatizadas comprueban la firma Ed25519 real, el rechazo de firmas modificadas, la serialización determinista, el campo minero sellado, el encadenamiento y la regla de préstamos. Una secuencia de tres bloques verifica que solo se reparta una recompensa por carrera, que el total llegue a 150 y que todos los hilos se detengan. Se comprueba que cada nonce observado pertenece al rango del nodo correspondiente.

La prueba obligatoria altera el contenido de un bloque minado sin cambiar su firma ni su hash: `es_valida()` devuelve falso. También se prueban cambios en hash, firma, hash anterior, número y minero. La interfaz incluye Alterar bloque 1 y Restaurar copia original para repetir la demostración. El video muestra tres carreras reales, sus ganadores y saldos, la cadena inválida tras una alteración y su restauración. Tiene subtítulos, sin audio; las carreras están resumidas y sus tiempos reales se indican en pantalla.

Fuente de requisitos: `guia_pow.pdf`, Dr. José de Jesús Ángel Ángel, Facultad de Ingeniería, Universidad Anáhuac México, pp. 1-3. Código, mediciones y capturas: ejecución local del proyecto.